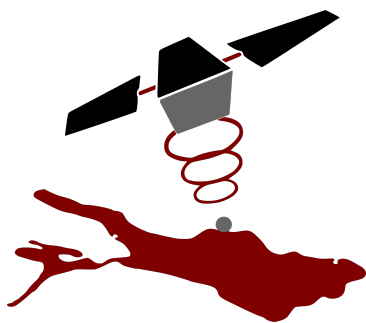


VHDL Development Guideline

for

Development of a telemetry data link for the
ERWIN cubesat



Project:	HW/SW Projekt TSL - 0004
Date:	June 8, 2026
Version:	1.2
Description:	This document aims to provide a guideline for VHDL development for the data link layer of the TM link for ERWIN

Revision History

Name	Date	Description	Version
Lukas Reil	14.10.2025	First Version, was still in Markdown	0.1
Lukas Reil	19.10.2025	Switched from Markdown to Latex	0.2
Lukas Reil	25.10.2025	Updated Chapter 2.4	1.0
Lukas Reil	16.05.2025	Updated Chapters 2.2 and 2.4	1.1
Lukas Reil	06.06.2025	Added Licence to header in chapter 2.1.2 and relaxed I/O signal types for entities, as well as updated example code in chapter 2.4	1.2

Contents

1	Introduction	1
1.1	Purpose	1
1.2	Scope	1
2	Standard	2
2.1	Structure	2
2.1.1	Project Structure	2
2.1.2	File Structure	2
2.2	Naming Convention	3

2.3	Code Formatting	3
2.4	Design Guidelines	4

1 Introduction

1.1 Purpose

This document aims to provide a consistent development guideline for everything to do with VHDL. This is to improve readability, as well as maintainability of code, written by multiple people.

1.2 Scope

This standard applies to all VHDL modules, testbenches and associated files developed for the data link layer of the TM link of ERWIN.

2 Standard

2.1 Structure

2.1.1 Project Structure

- All files relevant for synthesis shall be contained in or in a subfolder of a general **src** folder, from here on called 'source file'.
- All testbenches shall be contained in or in a subfolder of a general **tb** folder.
- All TCL scripts shall be contained in or in a subfolder of a general **scripts** folder.
- All steps of VHDL development shall be conducted using Vivado TCL scripts, where possible.

2.1.2 File Structure

- All source files shall contain exactly one entity and its one architecture, except for files describing packages.
- All source files shall have the same name as the contained entity, followed by a **.vhd** file extension.
- All source files shall have a header following Example 2.1.

```
1 -----
2 -- File: <FileName>.vhd
3 -- Created: <Date>
4 -- Author: <Author>
5 -- Project Name: <Project Name>
6 -- Description: <Short description>
7 -- License: https://github.com/rtnb/CCSDS-TM/blob/master/LICENSE
8 -----
```

Figure 2.1: VHDL File Header Template

2.2 Naming Convention

- All identifiers shall be named in lower snake case, if not specified otherwise.
- All architectures performing clocked logic shall be named behavioral.
- All architectures integrating instances of entities shall be named connectivity.
- All signals shall be named in lower snake case.
- Every signal not contributing to a bus interface (e.g. AXI-Stream) shall have one of the following suffixes, depending on the signal type:
 - `__i` for entity inputs
 - `__o` for entity outputs
 - `__r` for architecture internal signals, that get written to in a clocked process (from here on called 'registers')
 - `__s` for all other architecture internal signals
- All constants shall be named in caps lock with underscores to delimit words (EXAMPLE_CONSTANT).
- All generics shall follow the constants' naming scheme.
- All custom types shall have a `__t` suffix.
- All identifiers should be self explanatory.

```
1 entity dummy_data_producer is
2     port (
3         clk_i    : in  std_logic;
4         reset_i  : in  std_logic;
5         data_o   : out std_logic_vector(7 downto 0)
6     );
7 end entity dummy_data_producer;
```

Figure 2.2: VHDL Example Entity

2.3 Code Formatting

- Tabs shall be used as indentations.

-
- All VHDL keywords shall be used in lower case.
 - There shall only be one signal definition per line.
 - All entity port lists shall be indented as in Example 2.2.

2.4 Design Guidelines

- All input and output signals shall be of type **std_logic**, **std_logic_vector** or of a custom type.
- For entities exclusive to a module, which are not exposed to the top-level or a layer's top-level, the above restriction does not apply.
- All maths operations shall be performed, using **signed** or **unsigned** types, where possible.
- Math operations should not be performed with **integers**, except for array indices.
- An architecture that instantiates components shall not describe clocked logic.
- An architecture that instantiates components should only perform minimal sequential logic, like multiplexing.
- All registers shall be initialized with a predefined value.
- There shall be no unnamed constants (so called magic-numbers) in functional code, except for logic level checks (i.e. **signal_s = '0'**).
- All entities, architectures and processes shall be explicitly named.
- All named blocks shall be ended with its name explicitly mentioned.
- All code blocks shall be indented once more than their parent blocks.
- Every entity with a behavioral architecture shall define an active-low reset pin, which resets the entity asynchronously.

```
1 example_process : process(clk_i)
2 begin
3     if reset_n_i = '0' then
4         state_r <= IDLE;
5     elsif rising_edge(clk_i) then
6         state_r <= next_state_s;
7     end if;
8 end process example_process;
```

Figure 2.3: VHDL Example Process